

AI Traffic Management System

Real-World AI Capstone Project

Project Theme: Computer Vision + Adaptive Traffic Signal Control + Emergency Vehicle Priority

Abstract

The AI Traffic Management System is a smart-city capstone that uses computer vision to detect vehicles, estimate lane-wise traffic density and queues, and dynamically adjust traffic-signal timing. Instead of relying only on fixed timers, the system continuously observes traffic conditions and selects the next signal phase using measurable traffic demand. An emergency-vehicle module can prioritize ambulances, fire vehicles, or police vehicles by creating a safe green corridor. A dashboard presents live vehicle counts, congestion levels, signal state, alerts, and performance metrics. This architecture follows established research directions combining vehicle detection, tracking, traffic analytics, adaptive control, simulation, and dashboards.

Reference implementations commonly use YOLO-based detection, tracking, adaptive signal logic and SUMO simulation; advanced prototypes also combine computer vision with emergency pre-emption and real-time dashboards. ■cite■turn0search0■turn0search2■

1. Problem Statement

Conventional fixed-time signals may allocate green time inefficiently when traffic demand changes. A lightly loaded road can receive the same green duration as a heavily congested road, increasing waiting time and queue length. Emergency vehicles may also lose valuable time at intersections. The proposed system addresses these issues by converting camera/simulation data into traffic measurements and using those measurements to make adaptive control decisions.

2. Objectives

- Detect and classify vehicles from traffic-camera video.
- Count vehicles and estimate congestion for each approach/lane.
- Estimate queue length and waiting demand.
- Dynamically select signal phases and green duration.
- Detect emergency vehicles and activate priority logic.
- Detect selected violations such as red-light jumping when suitable camera geometry is available.
- Provide a real-time dashboard with traffic analytics.
- Compare adaptive control against fixed-time control using measurable KPIs.

3. Proposed System Architecture

Camera/Video → Frame Preprocessing → AI Vehicle Detection → Tracking → Lane & Queue Analytics → Adaptive Signal Decision → Traffic Signal Controller → Dashboard/Database. Emergency detection forms a priority branch that can override normal phase selection subject to safety rules.

4. Main Modules

Module	Purpose	Possible Technology
Video Input	Read CCTV/video/simulation frames	OpenCV, RTSP/video file
Vehicle Detection	Detect cars, buses, trucks, motorcycles and emergency vehicles	YOLO, PyTorch
Tracking	Maintain vehicle identities across frames	ByteTrack/DeepSORT
Traffic Analytics	Counts, density, queue length, speed estimates	Python, OpenCV
Adaptive Controller	Choose phase and green duration	Rule-based / optimization / RL
Emergency Priority	Create controlled green corridor	Python control logic
Dashboard	Live status, charts and alerts	Streamlit/React
Backend	APIs and real-time updates	FastAPI
Database	Store events and KPIs	SQLite/PostgreSQL
Simulation	Safe testing before deployment	SUMO

5. AI Workflow

1) Capture a frame. 2) Resize/normalize it. 3) Run the detector. 4) Track detections across frames. 5) Map detections to lanes/regions of interest. 6) Calculate vehicle count, density and queue length. 7) Calculate a demand score for each approach. 8) Select the phase with the highest safe priority. 9) Update signal timing. 10) Log metrics and display them on the dashboard.

6. Adaptive Signal Algorithm

A simple capstone-friendly demand score can be defined as: $\text{Demand} = w_1 \times \text{QueueLength} + w_2 \times \text{VehicleCount} + w_3 \times \text{WaitingTime} + w_4 \times \text{EmergencyPriority}$. The controller chooses the eligible phase with the highest score while enforcing minimum green, maximum green, amber and all-red safety intervals. This can later be upgraded to reinforcement learning.

7. Emergency Vehicle Logic

When an ambulance/fire/police vehicle is detected with sufficient confidence, the system identifies its approach and target intersection. It finishes or safely terminates the current phase, transitions through required clearance intervals, grants priority green, and then returns control to normal adaptive operation. In a real deployment, signal-controller safety interlocks and authorized emergency-preemption standards must remain in charge.

8. Dataset

Use traffic images/video containing varied vehicle types, camera angles, lighting, weather and road conditions. For a student prototype, a public vehicle dataset can be combined with locally captured or simulated frames where licensing permits. Annotate bounding boxes and classes such as car, bus, truck, motorcycle, ambulance, fire truck and police vehicle. Synthetic simulation data can help balance rare emergency classes.

9. Technology Stack

Python • OpenCV • YOLO/Ultralytics • PyTorch • NumPy • Pandas • ByteTrack/DeepSORT • FastAPI • Streamlit or React • SQLite/PostgreSQL • SUMO • Git/GitHub. A simpler version can use only Python + OpenCV + YOLO + Streamlit.

10. Hardware Requirements

Minimum prototype: laptop with 8 GB RAM, modern CPU, webcam/video file, and optional NVIDIA GPU. For real-time inference, a GPU or edge accelerator is recommended. For a college demonstration, recorded video plus simulation is safer and easier than controlling a real traffic signal.

11. Functional Requirements

- Accept uploaded traffic video or simulated camera feed.
- Detect and count vehicles in near real time.
- Display lane-wise traffic density.
- Calculate queue/waiting indicators.
- Change signal phase according to demand.
- Raise an emergency-vehicle alert.
- Show dashboard metrics and event logs.
- Export experiment results.

12. Non-Functional Requirements

The system should be responsive, fault-tolerant, explainable enough for operators, secure against unauthorized control, and auditable. Model confidence thresholds should be configurable. A fallback fixed-time or pre-approved signal plan should be available if the AI pipeline fails.

13. Database Design

Table	Example Fields
traffic_events	timestamp, junction_id, lane_id, vehicle_class, count, density
signals	junction_id, phase, start_time, end_time, mode
emergencies	timestamp, type, approach, confidence, action
experiments	scenario, controller, seed, avg_wait, queue, throughput
violations	timestamp, type, camera_id, confidence, evidence_path

14. Testing Plan

Test low, medium and high traffic scenarios. Compare fixed-time and adaptive modes using average waiting time, maximum queue length, throughput, travel time, number of phase changes, emergency response delay and inference FPS. Repeat each scenario several times with controlled seeds where simulation is used.

15. Expected Results

The expected outcome is lower average waiting time and shorter queues under changing demand, while maintaining safe phase transitions. Results should be reported from your actual experiments rather than copied from other projects. Existing capstone implementations demonstrate the feasibility of this architecture, including real-time vehicle detection, adaptive control, emergency preemption and simulation-based evaluation. ■cite■turn0search0■turn0search4■

16. Advantages

• Adapts to changing traffic demand • Uses existing camera concepts • Provides measurable analytics • Supports emergency priority • Can be tested safely in simulation • Can be extended to multiple junctions • Suitable for a live dashboard demonstration.

17. Limitations

Camera occlusion, night/rain conditions, poor camera placement, model bias, missed detections, network delays and incorrect lane calibration can affect performance. A student prototype should not directly operate public traffic signals without certified hardware, safety engineering, authorization and human oversight.

18. Future Scope

Multi-camera fusion, reinforcement-learning control, vehicle-to-infrastructure communication, pedestrian-aware control, route-level green-wave optimization, edge AI deployment, automatic incident detection, weather-aware control and integration with authorized smart-city traffic infrastructure.

19. 2-Minute Silent Demo Storyboard

0:00–0:15	Title + city intersection	Show project title and system architecture.
0:15–0:35	Normal traffic	Vehicles appear; AI boxes/counts them and displays lane density.
0:35–0:55	Congestion	One approach becomes crowded; dashboard shows rising queue and demand.
0:55–1:15	Adaptive signal	Signal switches to the high-demand approach; queue begins decreasing.

0:00–0:15	Title + city intersection	Show project title and system architecture.
1:15–1:35	Emergency vehicle	Ambulance is detected; emergency alert appears and priority phase activates.
1:35–1:50	Analytics	Show vehicle counts, average queue, signal state and throughput cards.
1:50–2:00	Conclusion	Show fixed-time vs adaptive comparison and future scope.

20. Viva / Presentation Questions

1. Why is fixed-time control insufficient? 2. How does YOLO detect vehicles? 3. How is traffic density calculated? 4. Why is tracking required? 5. How is green time selected? 6. How does emergency preemption work? 7. What metrics prove improvement? 8. Why should simulation be used before real deployment?

Conclusion

The AI Traffic Management System is a practical capstone that demonstrates an end-to-end perception-to-decision pipeline: cameras provide observations, AI converts them into traffic information, an adaptive controller makes a signal decision, and a dashboard explains the result. The project is suitable for a college demonstration because the full workflow can be reproduced using recorded video and SUMO without interacting with real public traffic infrastructure.

References

1. AI-based adaptive traffic management capstone architecture and evaluation examples. [■cite■turn0search0■](#)
2. Smart Traffic AI example using YOLO, tracking and adaptive signal control. [■cite■turn0search2■](#)
3. Adaptive traffic signal management using computer vision. [■cite■turn0search12■](#)